
Evaluate the Current System Landscape

Given a set of requirements, evaluate the authentication and authorization needs based on the system landscape.

Develop your career with Salesforce Training and Certification Preparation

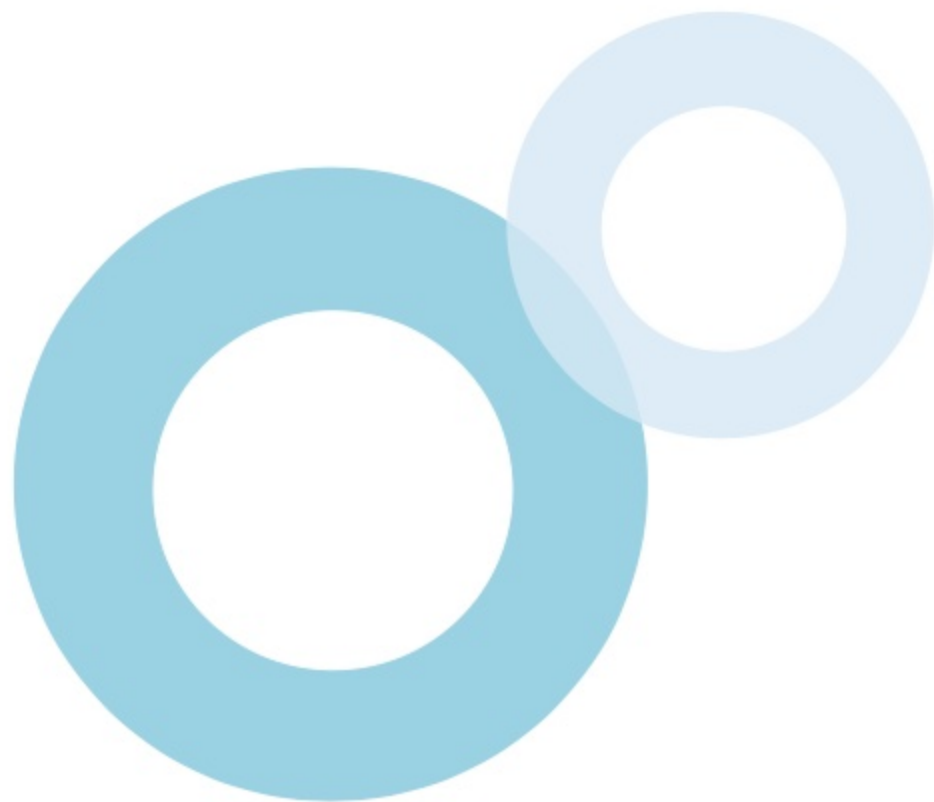
Table of Contents

-  [Named Credentials](#)
-  [OAuth 2.0](#)
-  [Connected Apps](#)
-  [External Client Apps](#)
-  [Certificates and Keys](#)
-  [Business Scenario](#)
-  [Requirements & Solutions](#)



After studying this topic, you should be able to:

-  Define tools to be used for authorization and authentication between Salesforce and external systems
-  Describe the use of named credentials, OAuth 2.0, Connected apps, certificates & keys
-  Determine the tools utilized to meet authorization and authentication requirements



Introduction

A **system landscape** represents an IT infrastructure and illustrates how servers, clients, external applications, and other systems relate with each other. System integration requires authentication and authorization to allow and secure communication between independent systems. Salesforce **supports different protocols** and **provides several tools** to allow various types of integration with their multi-tenant platform.

This section covers basic concepts related to **authorization and authentication** for external systems integrated with Salesforce, and describes, namely, the topics such as **Named Credentials**, **OAuth 2.0**, **Connected Apps**, and **Certificates & Keys**.



Overview

Named Credentials

Named credentials is a Salesforce tool designed to simplify the setup of authenticated callouts. In a named credential, the callout URL, required authentication parameters, and other settings can be specified.

Certificates and Keys

Salesforce certificates and key pairs are used to verify that a request is coming from a trusted organization. They are used when the org acts as an Identity Provider, or for encrypting communication with another system.



OAuth 2.0

OAuth, which is one of the authorization protocols supported by Salesforce, is an open-standard authorization protocol that is used to give client applications access to a protected resource by authorizing the client application using access tokens.

Connected Apps

A connected app is a Salesforce tool that allows the integration of an external application with Salesforce using standard APIs and protocols such as SAML, OAuth, and OpenID Connect.

Named Credentials

[Table of Contents](#) 

Legacy Named Credentials

Legacy named credentials allow specifying the **callout endpoint URL** and its **authentication parameters** in one location.

❖ AUTHENTICATION

The **authentication definition** is specified in the named credential and is **performed automatically** when used in a callout.

❖ MAINTENANCE

Callout endpoint URLs, authentication parameters, and other settings can be **easily updated** or **maintained**.

❖ SECURITY

User credentials are secured as they do not need to be **hardcoded**. Only the **API name** of the named credential is referenced in Apex.

❖ REMOTE SITE SETTING

When a URL is defined in a named credential, the domain **no longer needs** to be added in Remote Site Settings.

Note that **legacy named credentials will be deprecated** in a future release. Salesforce recommends using the **improved named credentials** made available in the Winter '23 release.

Identity Types

The **Identity Type** setting determines the way access to the external system is managed.

❖ ANONYMOUS

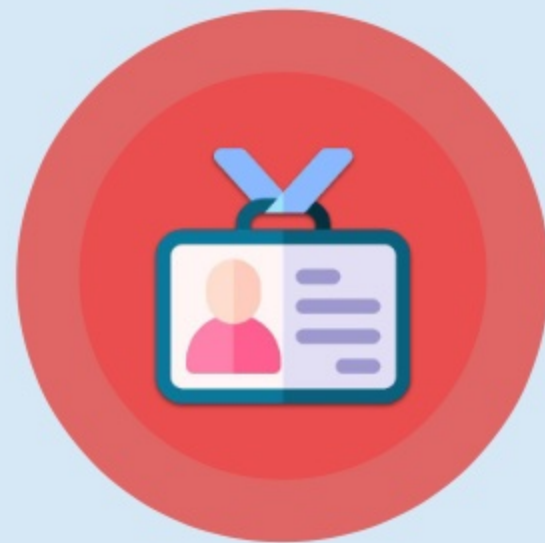
In this option, **no identity** is used, so **no authentication** is specified.

❖ PER USER

This option is used if access in the external system is restricted on a **per-user basis**. In this case, **separate credentials for each user** (user settings) are required in order to successfully perform the callout.

❖ NAMED PRINCIPAL

This option is used if the external system uses a **single user account** for all Salesforce org users. In this case, **the same set of credentials** is used **for all users** to access the external system when performing the callout.



Improved Named Credentials

Salesforce introduced the **next generation** of named credentials in Winter '23. The following describes some of the **features** in this improved version of named credentials.

❖ CUSTOM HEADERS

Custom name/value pairs can be defined and included in the HTTP callout as headers.

❖ EXTERNAL CREDENTIALS

Authentication details are stored separately in a component called external credentials which **can be reused** across different endpoints.

❖ PERMISSION SETS

Users can be granted to use the authentication defined in an **external credential** during a callout by assigning permission sets.

To allow users to create, edit, or delete named credentials and external credentials, the **Manage Named Credentials** permission can be assigned by enabling **“Allows users to modify Named Credentials and External Credentials.”** option in the profile or via a permission set.

Supported Callouts

Named credentials can be used in different types of **callout definitions**.

❖ APEX CALLOUTS

Apex can perform a callout to an endpoint that is defined in a named credential. For example:

```
req.setEndpoint('callout:MyNamedCredential/some_path');
```

❖ EXTERNAL SERVICES

A named credential that references an **external system** which provides **RESTful services** can be specified when creating an external service.

❖ EXTERNAL DATA SOURCES

A named credential can be used as the URL in an external data source for Salesforce Connect. For example:

URL field = **callout:MyNamedCredential/some_path**



Use Cases

The table below lists some **example** named credential use cases and relevant **identity types**.

Use Case	Identity Type
Providing service access to a third-party tool for compressing and optimizing images using a single account	Named Principal
Performing specific tasks remotely on an external site on behalf of a dedicated website administrator	Named Principal
Integrating with a back-end ordering system that authenticates access on a per-user basis	Per User
Sending expense data to an accounting system where users individually have their own username/password credentials	Per User
Requesting data from an external and publicly available service that provides currency exchange rate information	Anonymous

OAuth 2.0

[Table of Contents](#) 

OAuth

OAuth, which stands for **Open Authorization**, is an open-standard authorization protocol used to give client applications **access** to **protected resources** using **tokens**.

❖ OAUTH TOKENS

OAuth tokens represent **authorization permissions** given to an external application. A client application uses an **access token** to request access to a protected resource. If the access token is valid, the **client is granted access to the resource**.

❖ OAUTH FLOWS

There are several types of OAuth **authorization flows**. Which flow type to use depends on the type of application that needs to be integrated with Salesforce. Regardless of the flow type, there are **three main steps** involved in an authorization flow in general. First, a **client** application requests access to a protected resource. Second, an **authorization server** grants an access token to the client. Third, the **resource server** then grants the client access after successfully validating the token.



OAuth Tokens

OAuth authorization use different **types of tokens** depending on the type of **authorization flow** used. The following lists some of these tokens.

❖ AUTHORIZATION CODE

This is a **short-lived token** that is generated by the **authorization server** when a user **authorizes** a client application to access a resource on behalf of the user. This token is then submitted by the client application back to the authorization server in exchange for an **access token**.

❖ ACCESS TOKEN

A client application gains access to a protected resource by providing a **valid access token**. This token has a **longer lifetime** and can be **renewed** using what is called a **refresh token**, if available. Access tokens can also be **revoked** to invalidate access of clients who use the token.



OAuth Tokens

OAuth authorization use different **types of tokens** depending on the type of **authorization flow** used. The following lists some of these tokens.

❖ REFRESH TOKEN

A **refresh token** may be generated depending on the authorization flow used. This token can be used to **generate a new access token** when the access token has **expired** or has **been revoked**. Like access tokens, refresh tokens can also be revoked.

❖ ID TOKEN

An **ID token** may be issued by an authorization server. This token contains information about the **user**, **issue date** of the token, as well details of the **client application** requesting access to a resource.



OAuth Scopes

The **type of resources** that a client application has access to is determined by the **scopes** that were specified during the **authorization flow**.

❖ SCOPE

An external application accesses Salesforce as, what is called, a **connected app**. Scopes are **assigned** to a connected app. **Access to resources** in Salesforce is determined by the scopes associated with the connected app.

❖ TYPES

There are **several types** of scopes that can be assigned to a connected app. Scopes are **individually specified**, for example, to allow an external application to **manage details** of the current user, access to **all data that is accessible** to the current user, access to **Visualforce applications**, and more.



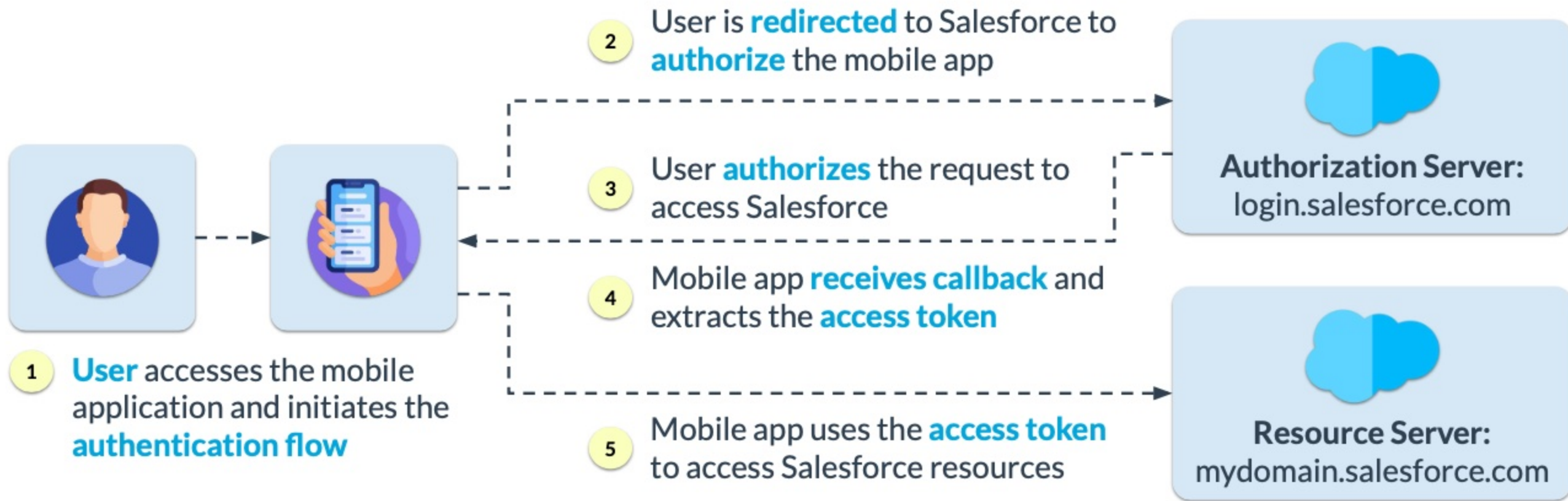
OAuth Flow Types

The table below lists and describes commonly-used **OAuth flows**.

OAuth 2.0 Flow	Description
Web Server Flow	The OAuth 2.0 web server flow is used to integrate Salesforce with an external web app . This flow generates an authorization code that will be exchanged for an access token.
User-Agent Flow	The OAuth 2.0 user-agent flow is used to integrate Salesforce with a desktop or mobile app . It could be used to integrate client applications that run in a browser. In this flow, an access token is generated directly from the authorization server .
JWT Bearer Flow	The OAuth 2.0 JWT bearer flow enables secure server-to-server authorization without any user login or manual interaction.
Client Credentials Flow	The OAuth 2.0 client credentials flow is used to authorize a client via a connected app using its consumer secret and consumer key .

OAuth Flow Example: User-Agent Flow

The example below illustrates an **authorization flow** that uses the **user-agent flow** to grant a **mobile application** access to a Salesforce resource.



OAuth Flow Example: Client Credentials Flow

The diagram below illustrates an **authorization flow** that uses the **client credentials flow** to grant an **external system** access to Salesforce resources. In this example, a cron job is used.



Connected Apps

[Table of Contents](#) 

Connected Apps

A connected app is used by an **external application** to access **Salesforce resources** on behalf of a **Salesforce user**.

❖ FRAMEWORK

A connected app serves as a framework to **authenticate**, **authorize**, or allow external applications to log in via **single sign-on** (SSO).

❖ PROTOCOLS

Connected apps support authentication protocols such as **SAML** (Security Assertion Markup Language), **OAuth**, and **OpenID Connect**.

❖ ACCESS CONTROL

Permissions and policies can be configured in a connected app that determines **which users only** have access to the connected app, and/or **from where** they can only use the connected app.



Connected Apps

A connected app is used by an **external application** to access **Salesforce resources** on behalf of a **Salesforce user**.

❖ TIMEOUT VALUE

A **timeout value** can be defined in a **connected app** that determines the **access token expiry** or **session timeout**. If not defined, the session timeout is based on the **user's profile**. If not defined in the user's profile, it is based on the **Session Settings** of the org in Setup.

❖ BLOCK ACCESS

A connected app can be blocked, which **terminates all active sessions** and **blocks all users** from accessing the app.

❖ REQUIRE MFA

Multi-factor authentication can be **activated** and **required** for all users who access the app.



Connected Apps

A connected app is used by an **external application** to access **Salesforce resources** on behalf of a **Salesforce user**.

❖ PERMITTED USERS

For an **OAuth-enabled** connected app, either all users are required to **authorize** the connected app the first time they access it, or users can be defined based on their **profile** or **permission sets** to have **pre-authorized** the connected app that manual authorization is “skipped”.

❖ IP RESTRICTIONS

Several options for IP restrictions are available that determines a user's **access to the connected app** and **use of refresh tokens** based on the IP address of the user.

❖ REFRESH TOKEN EXPIRY

A refresh token can be set as **valid unless revoked**, **invalid**, **expired if not used** for a certain duration, or **expired after** a certain duration.



Use Cases

Connected apps can basically be implemented in **4 main use cases**.

❖ DATA ACCESS

A connected app can be used to **request access** to Salesforce data **on behalf of a client application**. In this case, the connected app must be integrated with Salesforce using the **OAuth 2.0 protocol**.

❖ IDENTITY PROVIDER

A connected app can be used to **integrate a service provider** with the Salesforce org using **SAML 2.0** or **OpenID Connect**. In this integration, Salesforce acts as the identity provider.



Use Cases

Connected apps can basically be implemented in **4 main use cases**.

❖ THIRD-PARTY APPS

A third-party app is **installed and accessed** by users through a connected app. Access to the third-party app can be easily managed by setting **security policies** on the connected app.

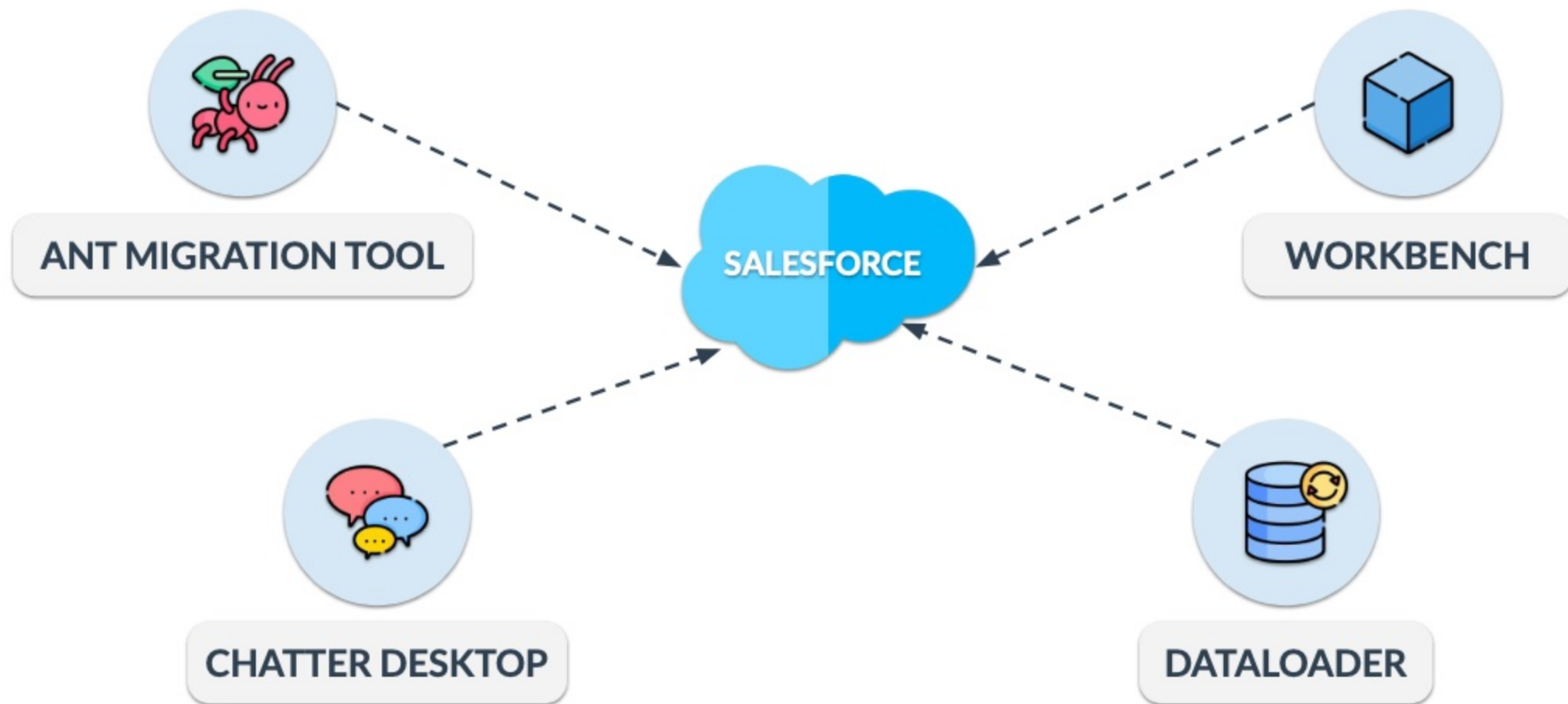
❖ AUTHORIZATION SERVER

Connected apps can be dynamically created using **OpenID Connect dynamic client registration**. In this integration scenario, Salesforce acts as the **OAuth authorization server** for protected resource that is hosted on an external system. A connected app is dynamically created for each **client application** that requests access after a successful authorization.



Standard Connected Apps

The following shows **external standard applications** that use **connected apps** to access or manage data as well as metadata in Salesforce.



External Client Apps

Table of Contents



External Client Apps

External client apps are the **next generation** of connected apps and are designed for **distribution** to other orgs via second-generation (2GP) **managed packaging**.

❖ THIRD-PARTY APPLICATIONS

Like connected apps, external client apps are created for **integrating** Salesforce with third-party applications.

❖ ENABLE EXTERNAL CLIENT APPS

In order to use create external client apps, the **Opt In to External Client Apps** setting in Setup must be enabled first.

❖ EXTERNAL CLIENT APP MANAGER

The External Client App Manager can be used to easily **create**, **manage**, and **update** external client apps from a user interface in Setup.



External Client Apps

❖ THIRD-PARTY APPLICATIONS

To view **OAuth consumer information** in an external client app, the user must be assigned the **View External Client App Consumer Secrets** permission.

❖ SALESFORCE DX PROJECT

To enable a Salesforce DX project to create or build an external client app, the project requires **additional configuration** in its files.



Certificates and Keys

[Table of Contents](#)



Certificates and Keys

When an org submits a request to another system, a **signature** can be used to affirm the **authenticity of the request**.

❖ SIGNATURE

Signatures use **certificates** and **key pairs**. Salesforce certificates defined with a key size of **2048 bits** or **3072 bits** expire in **one year** after creation. Salesforce certificates with a key size of **4096 bits** expire in **two years**.

❖ USE CASES

Salesforce certificates and key pairs are used to establish **authenticated SSL communications** with an external system. They are also used when the org acts as an **identity provider** to grant users access to **service providers**.

❖ VERIFICATION

Typically, a certificate and key pair needs to be **generated** when **verification is required** by the external system that the org is submitting a request to.



Certificates and Keys

When an org submits a request to another system, a **signature** can be used to affirm the **authenticity of the request**.

❖ SELF-SIGNED CERTIFICATE

A certificate can be **generated and self-signed** by Salesforce. This type of certificate is ideal for internal use and is more manageable as it can be **created any time**.

❖ CA-SIGNED CERTIFICATE

A certificate can be created in Salesforce using the **Create CA-Signed Certificate** option to generate a **Certificate Signing Request**, which is sent to a **certificate authority** (CA). Once signed by the CA, the certificate can be uploaded to Salesforce as **CA-signed certificate**, which can be used as a more authoritative and reliable way to affirm the authenticity of the requests coming from the org.



Certificates and Keys

When an org submits a request to another system, a **signature** can be used to affirm the **authenticity of the request**.

❖ MUTUAL AUTHENTICATION CERTIFICATE

Two communicating parties can be required to verify each other's identity, which is also known as **2-way SSL authentication**, by using a mutual authentication certificate. After a certificate has been uploaded, the “**Enforce SSL/TLS Mutual Authentication**” user permission needs to be enabled for an “**API Only**” user. When enabled, mutual authentication is also enforced for API login requests that generate new user sessions. Note that **Salesforce support** needs to be contacted to enable the org feature and allow uploading a mutual authentication certificate.



Example Use Cases

The following describes using **certificates** in a given use case.

Use Case	Solution
Requiring Salesforce and an external system to verify each other's identity before exchanging sensitive data	❖ In Salesforce, a mutual authentication certificate is used ❖ In the external system, a trusted CA-signed certificate is used
Using certificates to encrypt communication in a private network for testing internal applications	❖ A self-signed certificate can be used

Business Scenario

Table of Contents 

Business Scenario: Part 1

Read the business scenario below and determine the most appropriate solutions for the company's requirements.



Global Pharmaceutical is a company that sells and distributes medical products worldwide. The European division, for example, has headquarters in 10 countries including Spain, France, Germany and others. It has more than 3,000 medical sales representatives who visit 30,000+ medical facilities on a regular basis.

Global Pharmaceutical uses Salesforce as its primary CRM for managing accounts, contacts, and opportunities. Sales reps use Salesforce when meeting with clients, recording visits, creating sales orders, and signing contracts with clients.

Business Scenario: Part 2

Read the business scenario below and determine the most appropriate solutions for the company's requirements.



When visiting medical facilities such as a hospital, clinic, or pharmacy to meet a client, a sales rep typically accesses the org using a laptop. However, not all locations allow access to the internet. In this case, the sales rep records any information using a digital notepad, for example, or a physical one when a laptop is not available. Once internet is accessible, data can then be manually entered in Salesforce.

The Chief Technical Officer (CTO) wants to have a mobile app developed that supports tablets and phones on iOS, as well as an offline mode feature. The offline mode feature would allow sales reps to enter data which will be automatically uploaded to Salesforce when an internet connection is available.

Requirements & Solutions

[Table of Contents](#)



Requirement 1



SCENARIO

The medical sales representatives of the company should use the mobile application when meeting with clients. The sales reps should log into the mobile application once, using their Salesforce credentials to grant the application access to Salesforce on behalf of their user accounts. The mobile application should be integrated with Salesforce in a secure way such that only sales reps should be allowed to access Salesforce through the app.



Solution 1



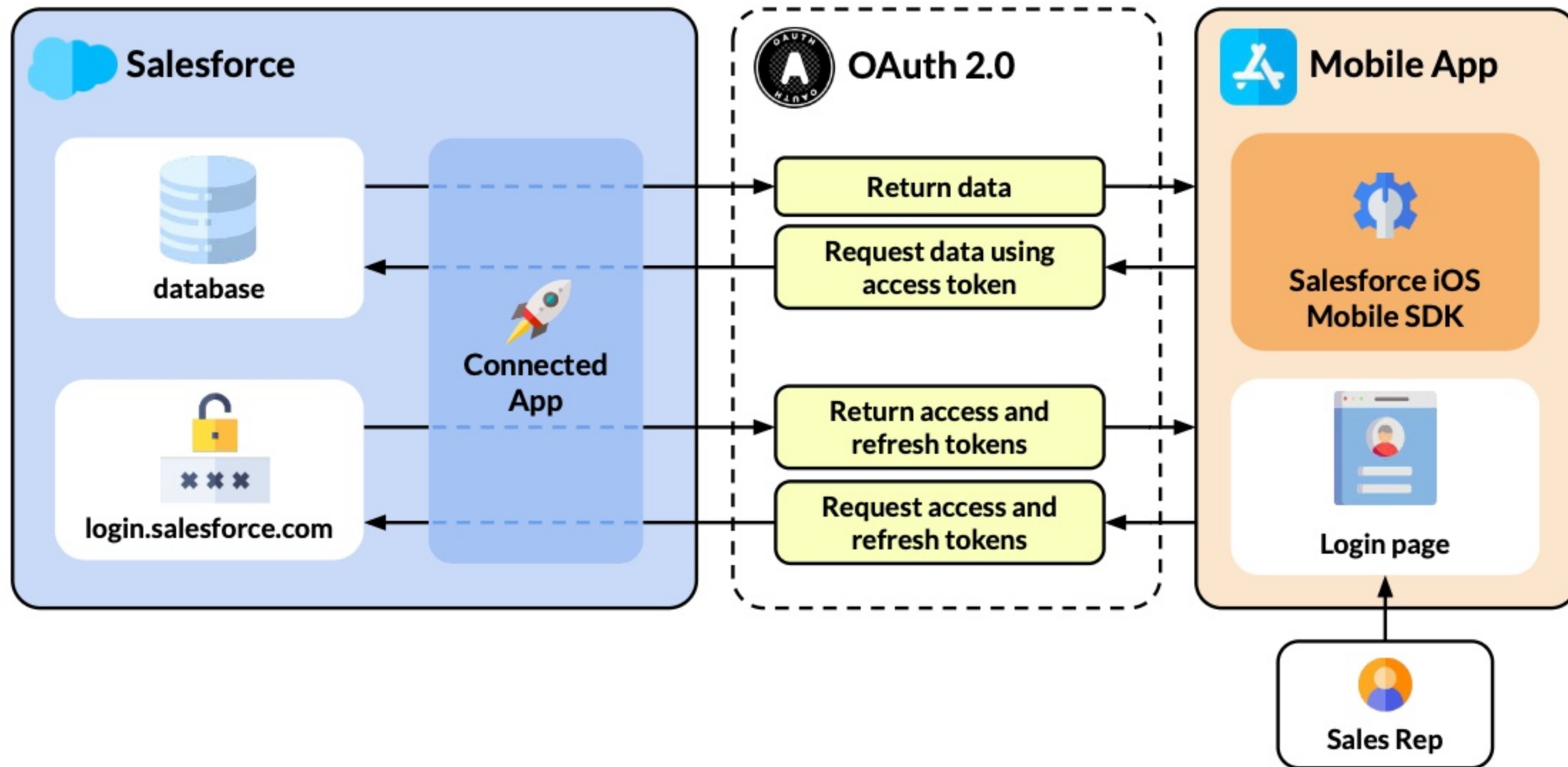
SOLUTION

A sales rep can authorize the mobile app access to Salesforce using the **OAuth 2.0 User-Agent Flow**, which is the recommended authorization flow for integrating mobile applications. When accessing the mobile app the first time, users are required to enter their credentials to generate an **access token**. The mobile app extracts the access token from the callback and uses it to access Salesforce.

A **connected app** is created that will be used by the mobile app to **authorize** and **authenticate** API requests sent to the org. Using the **OAuth policy settings**, user access can be defined based on **profiles** and **permission sets**.



Solution 1



Requirement 2



SCENARIO

The company would also like to use the mobile app to allow recording data that contains highly sensitive healthcare information which will be used for research & development by the company.

To ensure high confidentiality and security in the communication between Salesforce and the mobile application, the CTO would like to establish mutual authentication and eliminate the possibility of impersonation in the network.



Solution 2



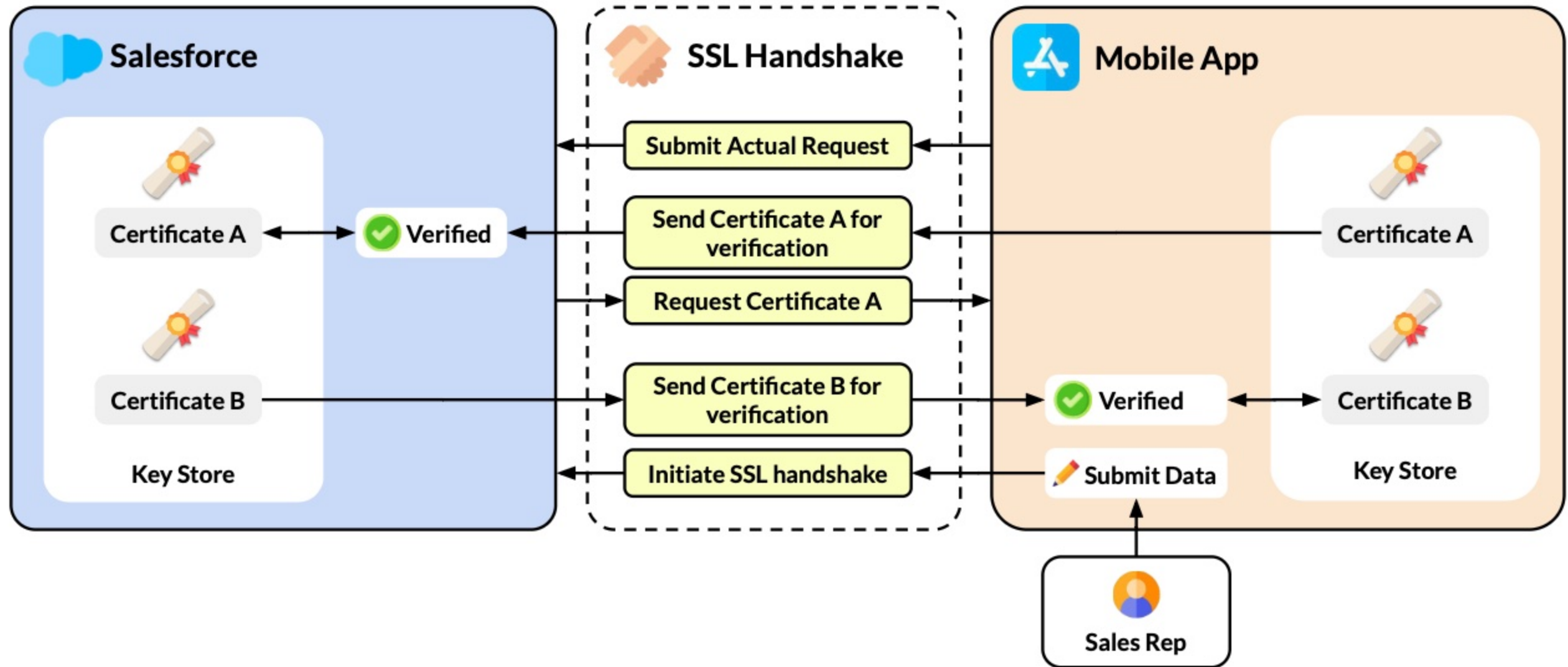
SOLUTION

In **mutual authentication**, or also known as **two-way SSL authentication**, the client and server verifies each other's identities (**SSL handshake**) before any actual request is processed. For example, a **mutual authentication certificate** can be used in Salesforce. Then, a **CA-signed certificate** can be used by the mobile app.

In an SSL handshake, Salesforce (server) sends its certificate to the mobile app (client) for verification. After the mobile app validates the certificate, it also sends its certificate to Salesforce for its turn to be verified. If both **validations are successful**, then requests can be processed.



Solution 2



Learn More

-  [Named Credentials](#)
-  [Named Credentials as Callout Endpoints](#)
-  [Authorize Apps with OAuth](#)
-  [OAuth Authorization Flows](#)
-  [Connected Apps](#)
-  [Build Integrations Using Connected Apps](#)
-  [Certificates and Keys](#)